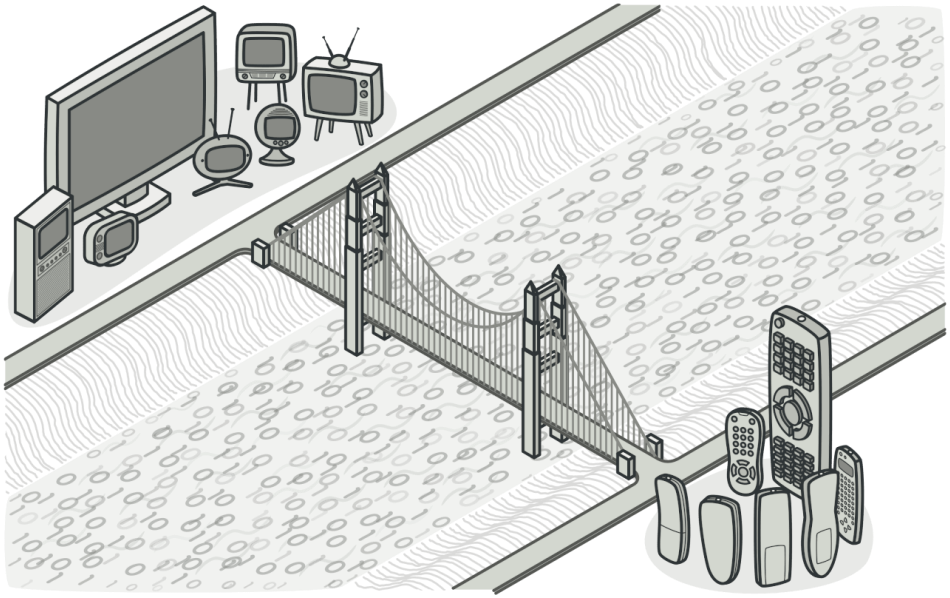
Структурные паттерны. Мост



Паттерн Мост
Bridge

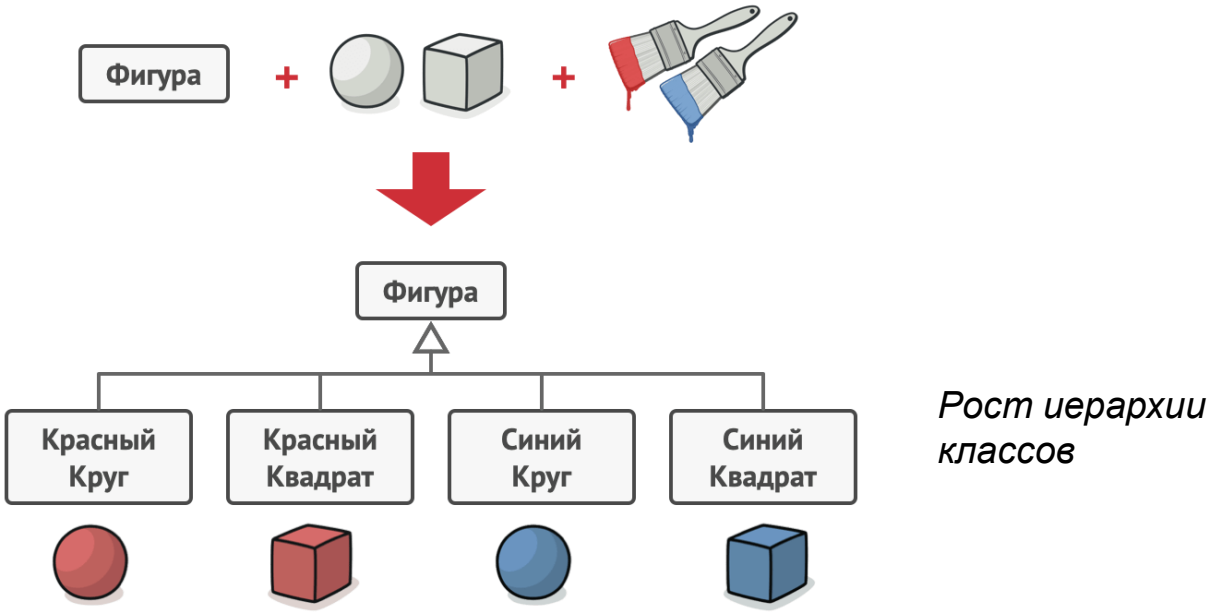


Мост — это структурный паттерн проектирования, который разделяет один или несколько классов на две отдельные иерархии — *абстракцию* и *реализацию*, позволяя изменять их независимо друг от друга.

Проблема

Абстракция – собирательное понятие дающее возможность просто и обобщенно представить сложную сущность, состоящую из множества частей.

Реализация (конкретика)– точное понятие, описывающее единичный (не требующий разложения на более мелкие части) объект с ясно определенными признаками.



Рост иерархии классов

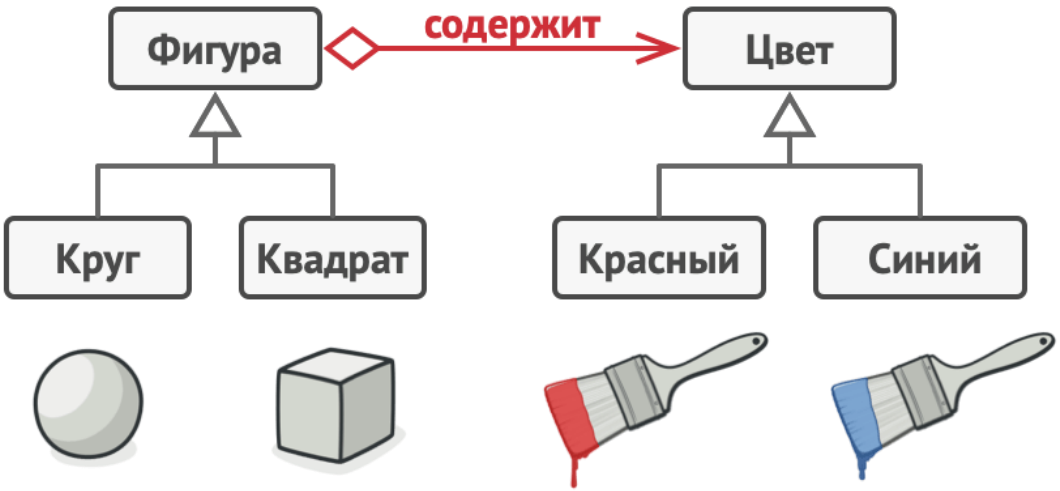
Если некоторая абстракция может иметь несколько возможных реализаций, то обычно применяют наследование.

Абстрактный класс определяет интерфейс абстракции, а его конкретные подклассы по-разному реализуют его.

Наследование жестко привязывает реализацию к абстракции, что затрудняет независимую модификацию, расширение и повторное использование абстракции и ее реализации.

Решение

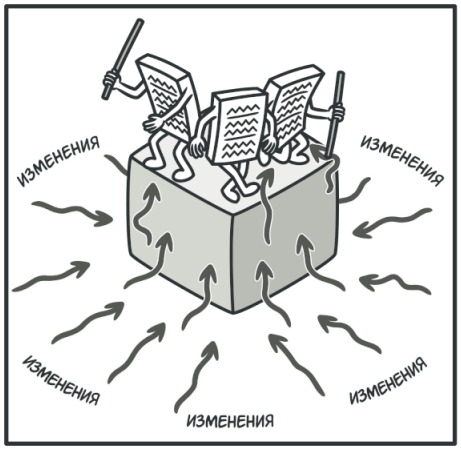
Паттерн ***Мост*** предлагает заменить *наследование - агрегацией* или *композицией*. Для этого нужно выделить одно из понятий в отдельную иерархию и ссылаться на объект этой иерархии, вместо хранения его состояния и поведения внутри одного класса.

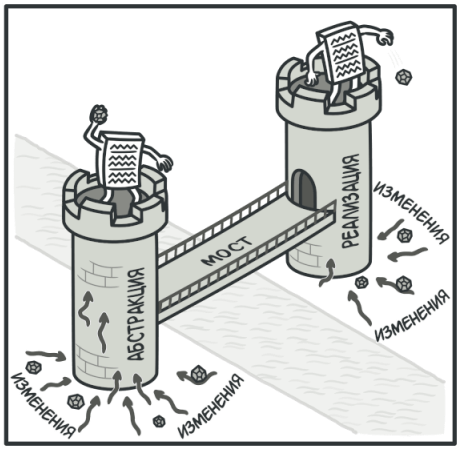


Абстракция — это образный слой управления чем-либо.
Он не выполняет работу самостоятельно, а делегирует её слою ***реализации***.

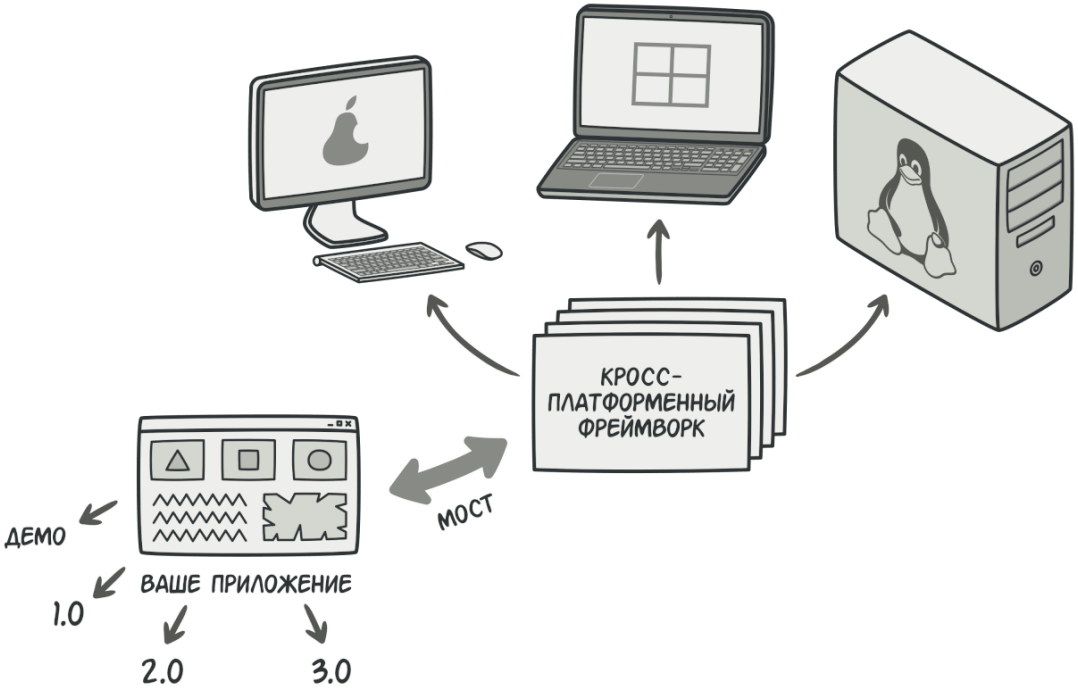
Решение

Абстракция - слой графического интерфейса приложения.
Реализация - слой взаимодействия с операционной системой.





Абстракция делегирует работу одному из объектов *реализаций*.
Реализации можно взаимозаменять, но только при условии, что все они будут следовать общему интерфейсу.

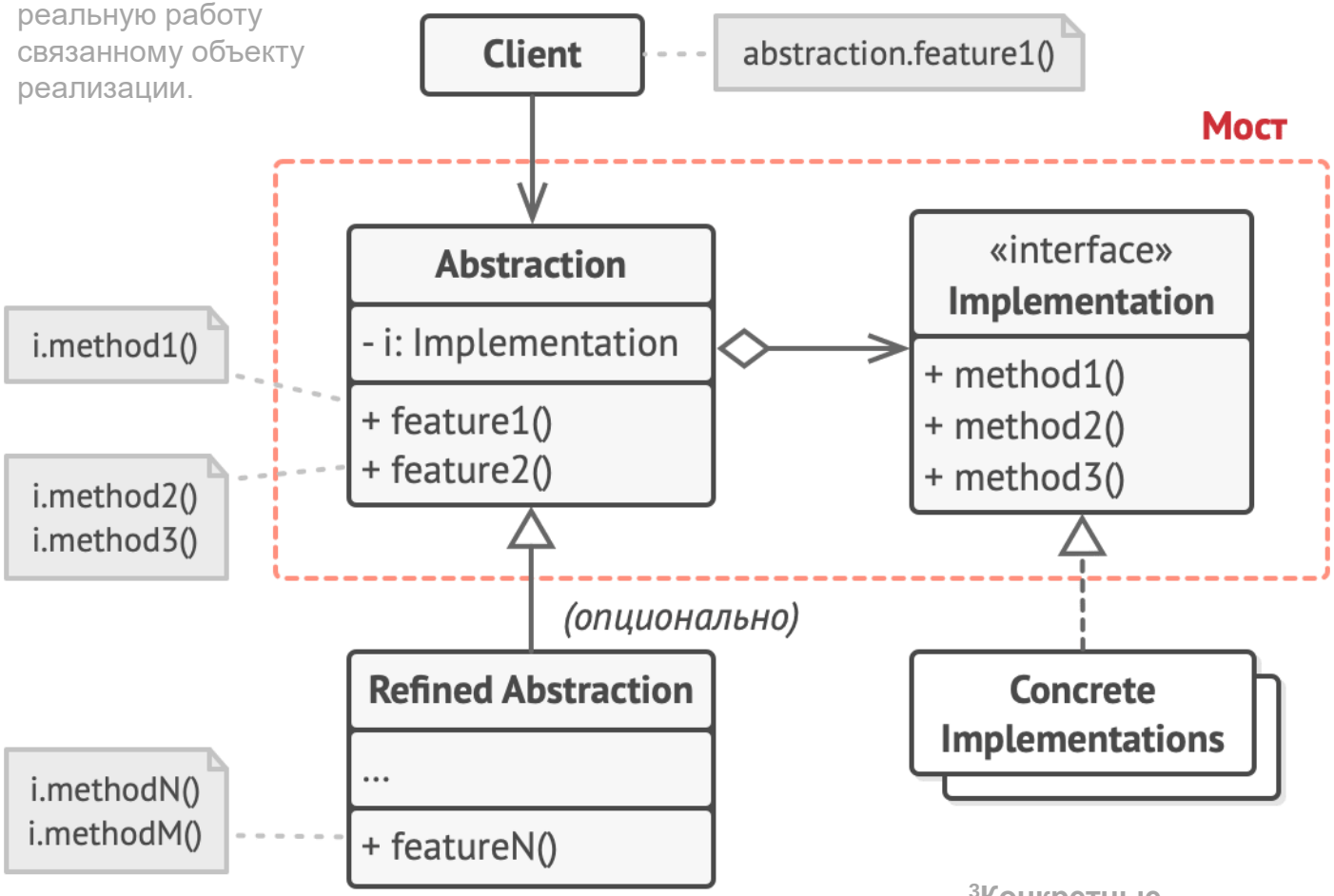


- Можно изменять графический интерфейс приложения, не затрагивая низкоуровневый код работы с операционной системой.
- Можно добавлять поддержку новых операционных систем, создавая подклассы реализации, без необходимости менять классы графического интерфейса.

Структура паттерна

[1]**Абстракция** содержит управляющую логику. Код абстракции делегирует реальную работу связанному объекту реализации.

[5]**Клиент** работает только с объектами абстракции. Не считая начального связывания абстракции с одной из реализаций, клиентский код не имеет прямого доступа к объектам реализации.



[2]**Реализация** задаёт общий интерфейс для всех конкретных реализаций. Все методы, которые здесь описаны, будут доступны из класса абстракции и его подклассов.

Интерфейсы абстракции и реализации могут как совпадать, так и быть совершенно разными..

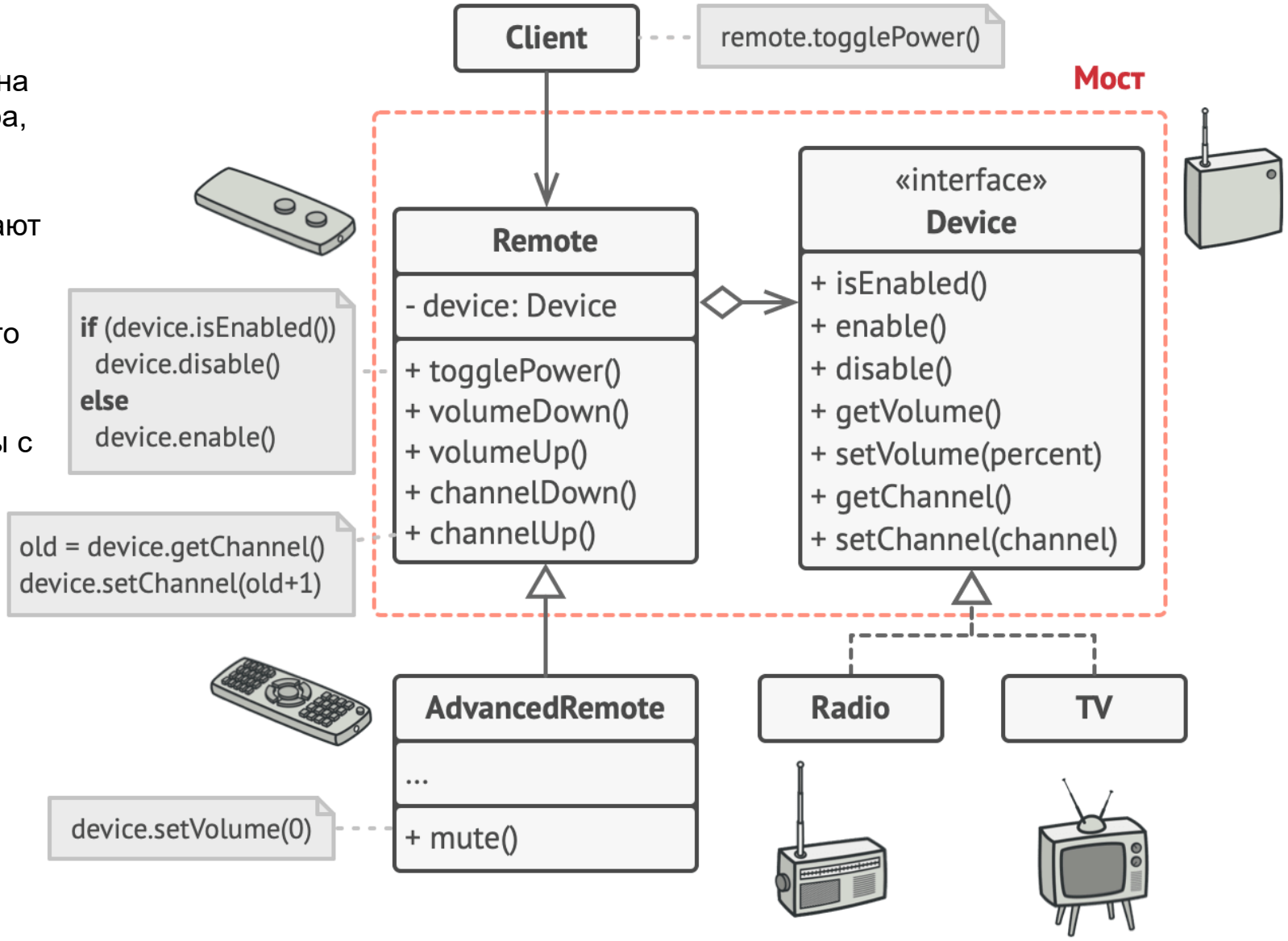
[4]**Расширенные абстракции** содержат различные вариации управляющей логики. Как и родитель, работает с реализациями только через общий интерфейс реализации.

[3]**Конкретные реализации** содержат платформо-зависимый код.

Структура паттерна. Пример 1

Мост разделяет монолитный код приборов и пультов на две части: приборы (выступают реализацией) и пульты управления ими (выступают абстракцией).

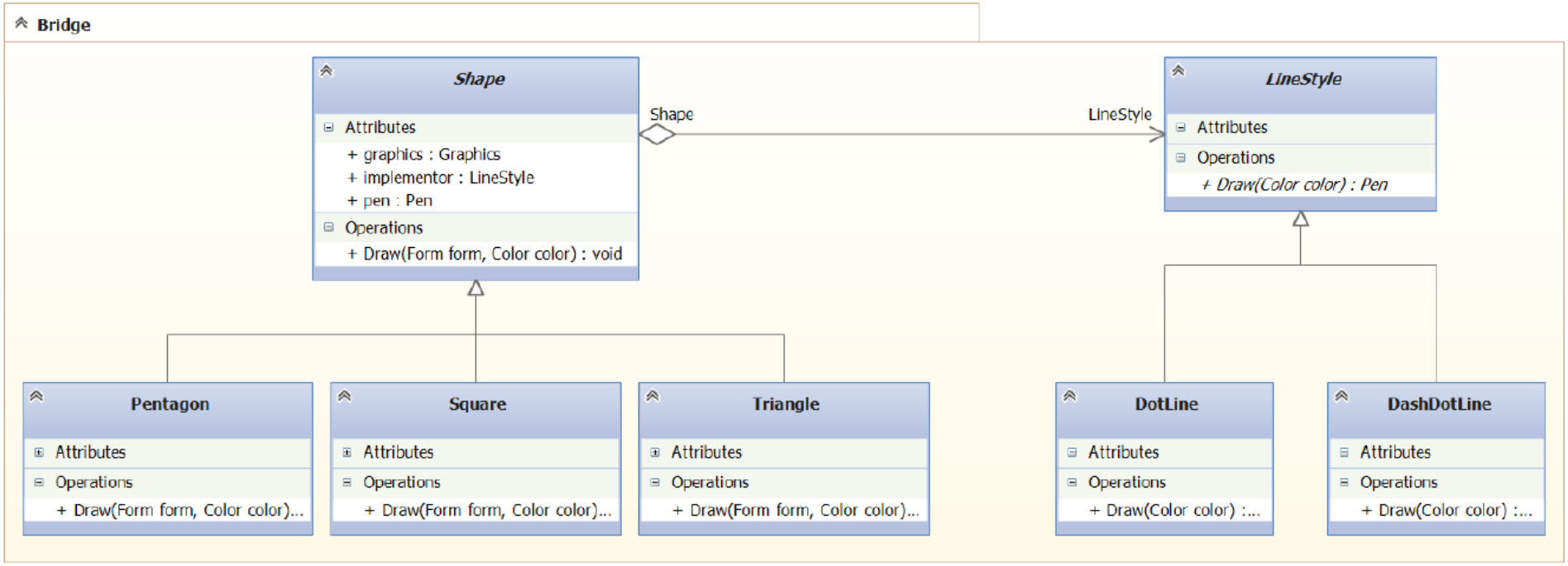
Класс пульта имеет ссылку на объект прибора, которым он управляет. Пульты работают с приборами через общий интерфейс. Это даёт возможность связать пульты с различными приборами.



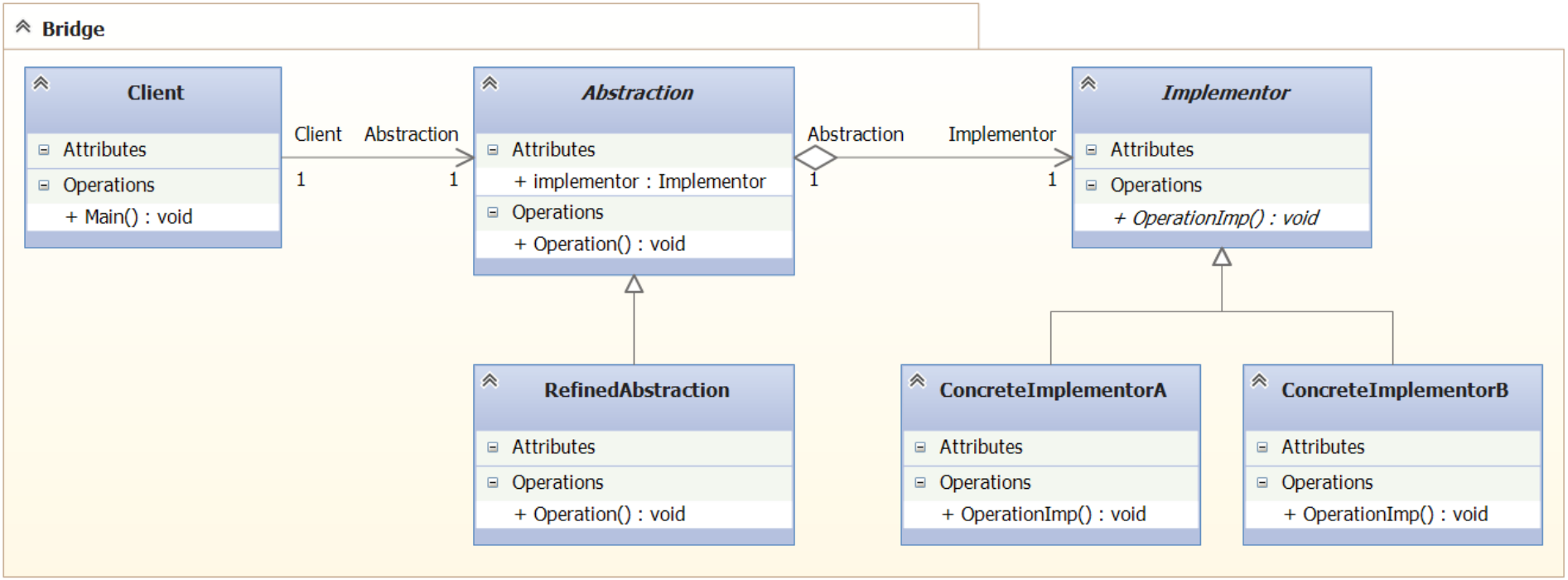
Структура паттерна. Пример 2

Диаграмма классов графического редактора:

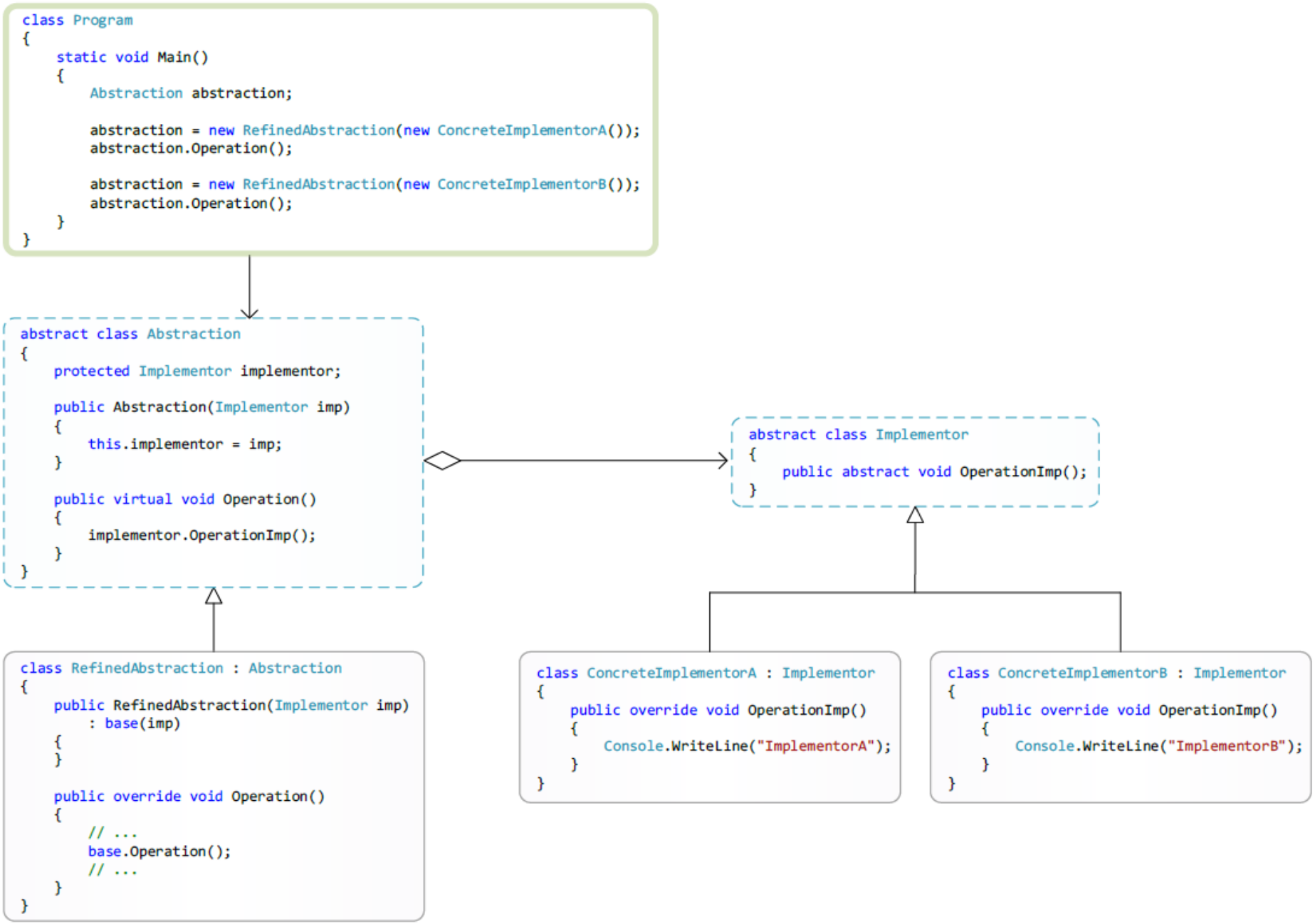
- классы геометрических форм **Shape** (абстракций)
- классы типов линий **LineStyle** (реализаций).



Структура паттерна UML



Структура паттерна на языке C#



Участники

Abstraction - Абстракция:
Предоставляет интерфейс для абстракции. Хранит ссылку на Implementor.

RefinedAbstraction - Уточненная абстракция:
Расширяет интерфейс, предоставляемый абстракцией.

Implementor - Реализатор:
Предоставляет интерфейс для реализации. Чаще всего класс Implementor предоставляет низкоуровневый интерфейс, а Abstraction предоставляет высокоуровневый интерфейс.

ConcreteImplementor - Конкретный реализатор:
Реализует интерфейс класса Implementor.

Отношения между участниками

Отношения между классами

Абстрактный класс Abstraction связан связью отношения агрегации с абстрактным классом Implementor.

Конкретный класс RefinedAbstraction связан связью отношения наследования с абстрактным классом Abstraction.

Конкретные классы-реализаторы ConcreteImplementorA и ConcreteImplementorB связаны связью отношения наследования с абстрактным классом Implementor.

Отношения между объектами

Объекты типа Abstraction перенаправляют запросы клиента объектам типа Implementor.

Применение

Когда необходимо разделить монолитный класс, который содержит несколько различных реализаций некоторой функциональности (например, если класс может работать с разными системами баз данных).

Изменения, вносимые в одну из реализаций, приводят к редактированию всего класса, что может привести к внесению случайных ошибок в код.

Мост позволяет разделить монолитный класс на несколько отдельных иерархий. После этого можно менять их код независимо друг от друга. Это упрощает работу над кодом и уменьшает вероятность внесения ошибок.

Шаги реализации

1. **Определить, существует ли в классах непересекающиеся применения**. Это может быть *функциональность/платформа*, *предметная-область/инфраструктура*, *фронт-энд/бэк-энд* или *интерфейс/реализация*.
2. **Продумать, какие операции** будут нужны клиентам, и описать их в базовом классе абстракции.
3. **Определить поведения, доступные для всех применений**, и выделить из них ту часть, которая нужна абстракции. На основании этого описать общий интерфейс реализации.
4. **Для каждого применения создать свой класс конкретной реализации**. Все они должны следовать общему интерфейсу, который им выделен перед этим.
5. **Добавить в класс абстракции ссылку на объект реализации**. Реализовать методы абстракции, делегируя основную работу связанному объекту реализации.
6. **Если есть несколько вариаций абстракции**, создать для каждой из них свой подкласс.
7. **Клиент должен подать объект реализации в конструктор абстракции**, чтобы связать их воедино. После этого он может свободно использовать объект абстракции.

Преимущества и недостатки

+ Позволяет строить платформо-независимые программы.

+ Скрывает лишние детали реализации от клиентского кода.

+ Реализует принцип *открытости/закрытости*.

- Усложняет код программы из-за введения дополнительных классов.